CODER HOUSE

Proyecto Final Data Science

“Modelo de clasificación para la detección temprana de enfermedades coronarias”

Entrega final

- Integrantes: Lucas Orden Guiñazu
  Manuel Demarchi Goitia
  Marcelo Ferraro
- Comisión: 23420
- Grupo 12
- Docente: Norberto González
- Tutor: Franco Tartaglia

2022

Índice

Sobre nosotros **4**

Problemática a resolver **4**

Solución propuesta **4**

Modelado 5

Data Acquisition 5

Descripción de variables y dominio 5

Estadística descriptiva **6**

El dataset obtenido de la base de datos posee 918 filas y 12 columnas, de las cuales 11 (once) son variables independientes a analizar y 1 (una) corresponde a la variable target o dependiente. **6**

Figura 1: Información del dataset. **7**

Es posible visualizar en la figura n°3 que ciertas variables numéricas continuas (como Oldpeak, RestingBP y Cholesterol) poseen valores absurdos o irreales en el dataset. Las mismas serán tratadas posteriormente. **8**

Data Wrangling y EDA **8**

Variables categóricas ordinales 8

Orden de preponderancia 8

Variables numéricas 8

Correlación sin encodear (variables numéricas) 10

Feature Engineering **11**

Correlación con variables encodeadas 12

Baseline **13**

Test - Train - Validation split 13

Métricas 13

Conclusiones primera iteración **14**

Segunda iteración **15**

Modelos a implementar 15

Resultados obtenidos y comparativa de métricas 16

Función de pérdida sobre Train y Val 16

Accuracy sobre Train, Val y Test 16

Recall sobre Val y Test (clase 1) 17

ROC/AUC sobre Test incluyendo umbrales 17

Árbol de decisión 17

Random Forest 19

KNN 19

Regresión Logística 21

CODER HOUSE

Análisis PCA 22
Corrección del umbral (optimal cutoff) 23
Mejoras para iteraciones posteriores 24

Conclusiones segunda iteración 24

Tercera iteración 26
Modelos optimizados y métricas 26
GridSearchCV utilizando StratifiedKFold 26
Árbol de decisión 26
Random Forest 27
KNN 27
Regresión Logística 28
RandomizedSearchCV utilizando StratifiedKFold 28
Modelos boosting 29
Modelos boosting optimizados 31
Bagging (Bootstrap aggregating) 32
Bagging optimizado 34

Conclusiones tercera iteración 35

Conclusiones finales 37

CODER HOUSE

Sobre nosotros

Somos una consultora independiente que brinda soluciones end-to-end a problemas de Inteligencia artificial (desde el desarrollo de modelos, testeo, puesta en producción y mantenimiento) dentro de la órbita de Shallow Learning, Deep Learning y Computer Vision.

Problemática a resolver

Luego de persistir los datos de cientos de historias clínicas, el departamento de Cardiología de un nosocomio nos contactó para ver si es posible predecir e inferir si una persona es propensa a padecer una enfermedad cardíaca basándonos en los resultados de estudios considerados “de rutina”, logrando - además de una detección en etapa temprana de la afección-, un abaratamiento de costos que pueden ser destinados a renovar el equipamiento médico.

Se nos aclaró que es menester que el modelo a desarrollar no solo sea performante a la hora de realizar una predicción, sino también que el mismo ***disminuya lo máximo posible la cantidad de Falsos negativos (FN)***, entendiendo que es mejor diagnosticar tempranamente que una persona padece una enfermedad y equivocarse en los sucesivos estudios, que no hacerlo y la misma sea más difícil de tratar en una etapa tardía.

Solución propuesta

Se propone realizar un modelo de clasificación binaria, realizando previamente un análisis de los datos persistidos en la base, tanto de la integridad de los mismos como de su poder predictivo para la variable target a inferir. Se seleccionará aquel modelo que no solo brinde el mayor ***accuracy*** sobre el set de validación, sino que se prestará especial atención en el valor del ***recall*** buscando que el mismo tienda a 1 (uno) ó -lo que es lo mismo- que los falsos negativos tiendan a cero.

$$Recall = TP / (TP + FN)$$
$$Accuracy = TP + TN / (TP + TN + FN + FP)$$

Modelado

Data Acquisition

Una vez obtenido el acceso a la base, se volcó el contenido de la misma a un archivo .csv el cuál puede obtenerse aquí[1]. El nosocomio en cuestión nos brindó una descripción de las variables presentes en el mismo, como así también el dominio o rango esperado de sus variables numéricas, en caso de que haya algún tipo de error en la base de datos. En una primera etapa se le prestará especial atención a aquellas variables que contengan valores considerados absurdos e imposibles para un ser humano.

Descripción de variables y dominio

- **Age**: edad del paciente medida en años **[29,77]**.
- **Sex**: sexo del paciente **[M: Hombre, F: Mujer]**.
- **ChestPainType**: tipo de dolor de pecho **[TA: Angina Típica, ATA: Angina Atípica, NAP: Dolor no proveniente de Angina, ASY: Asintomático].**
- **RestingBP**: presión sanguínea en reposo medida en [mm Hg] **[94, 200]**.
- **Cholesterol**: colesterol calculado mediante proporciones de HDL, LDL y triglicéridos medido en [mm/dl] **[126, 564].**
- **FastingBS**: azúcar en sangre en ayuno **[1: si FastingBS > 120 mg/dl, 0: si no lo es]**

[1] El dataset en cuestión fue obtenido de Kaggle, pero a propósitos del storytelling, este dato no es mencionado.

CODER HOUSE

- **RestingECG**: resultado de electrocardiograma en reposo **[Normal: Normal, ST: anormalidad en la onda ST-T (onda T invertida y/o elevación o depresión ST > 0.05 mV), LVH: evidencia de probable o definitiva hipertrofia ventricular izquierda usando el criterio de Estes].**

- **MaxHR**: máximo ritmo cardíaco alcanzado **[60, 202]**.

- **ExerciseAngina**: ejercicio indujo angina **[Y: Si, N: No]**.

- **Oldpeak**: depresión de ST inducida por ejercicio relativa al reposo **[0, 6].**

- **ST_Slope**: pendiente alcanzada durante ejercicio del segmento ST **[Up: pendiente positiva, Flat: pendiente nula, Down: pendiente negativa].**

- **HeartDisease**: variable target a predecir **[1: posee una enfermedad coronaria, 0: no posee una enfermedad coronaria].**

Estadística descriptiva

- El dataset obtenido de la base de datos posee ***918 filas*** y ***12 columnas***, de las cuales ***11 (once)*** son variables independientes a analizar y ***1 (una)*** corresponde a la variable target o dependiente.

- El dataset no posee valores nulos ni filas repetidas.

- De las 11 (once) variables independientes, 5 (cinco) son **categóricas** (Sex, ChestPainType, RestingECG, ExerciseAngina, ST_Slope) y 6 (seis) son **numéricas** (Age, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak). La variable target (HeartDisease) ya se encuentra binarizada.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    object
 2   ChestPainType   918 non-null    object
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    object
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    object
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    object
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
```

Figura 1: Información del dataset.

	count	unique	top	freq
Sex	918	2	M	725
ChestPainType	918	4	ASY	496
RestingECG	918	3	Normal	552
ExerciseAngina	918	2	N	547
ST_Slope	918	3	Flat	460

Figura 2: Estadística descriptiva básica de las variables categóricas.

	count	mean	std	min	25%	50%	75%	max
Age	918.0	53.510893	9.432617	28.0	47.00	54.0	60.0	77.0
RestingBP	918.0	132.396514	18.514154	0.0	120.00	130.0	140.0	200.0
Cholesterol	918.0	198.799564	109.384145	0.0	173.25	223.0	267.0	603.0
FastingBS	918.0	0.233115	0.423046	0.0	0.00	0.0	0.0	1.0
MaxHR	918.0	136.809368	25.460334	60.0	120.00	138.0	156.0	202.0
Oldpeak	918.0	0.887364	1.066570	-2.6	0.00	0.6	1.5	6.2
HeartDisease	918.0	0.553377	0.497414	0.0	0.00	1.0	1.0	1.0

Figura 3: Estadística descriptiva básica de las variables numéricas.

CODER HOUSE

Es posible visualizar en la figura n°3 que ciertas variables numéricas continuas (como Oldpeak, RestingBP y Cholesterol) poseen valores absurdos o irreales en el dataset. Las mismas serán tratadas posteriormente.

Data Wrangling y EDA

Variables categóricas ordinales

El nosocomio nos informó que de las 5 (cinco) variables categóricas, 3 (tres) son importantes de ser distinguidos sus valores por orden de preponderancia a la hora de auscultar y analizar (ChestPainType, RestingECG y ST_Slope). Las restantes son solo clasificaciones que pueden considerarse **variables categóricas nominales** susceptibles de ser binarizadas.

Orden de preponderancia

ChestPainType: asintomático < dolor no proveniente de angina < angina atípica < angina típica.

RestingECG: normal < anormalidad en la onda ST-T < evidencia de probable o definitiva hipertrofia ventricular izquierda usando el criterio de Estes.

ST_Slope: pendiente positiva < pendiente nula < pendiente negativa.

Ambos tipos de variables categóricas serán encodeadas adecuadamente a la hora de realizar la ingeniería de variables correspondiente.

Variables numéricas

Se dispuso a graficar la distribución y el boxplot de cada una, habiendo filtrado previamente los valores absurdos o irreales encontrados mediante el análisis descriptivo. Se adjuntan gráficos de aquellas que posean outliers y su posterior interpretación.

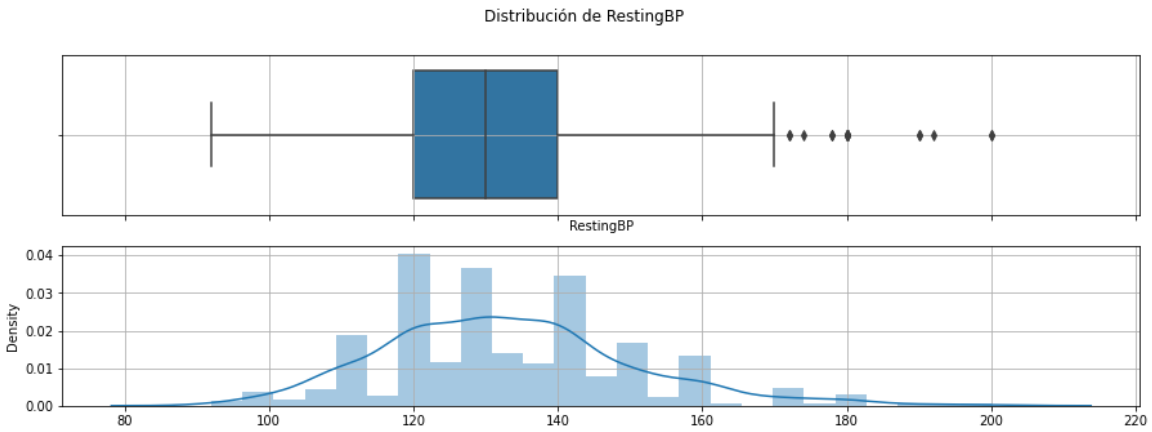


Figura 4: Distribución de la variable presión sanguínea en reposo.

Si bien existen outliers para la variable RestingBP, los mismos se encuentran dentro del dominio esperado y no serán imputados.

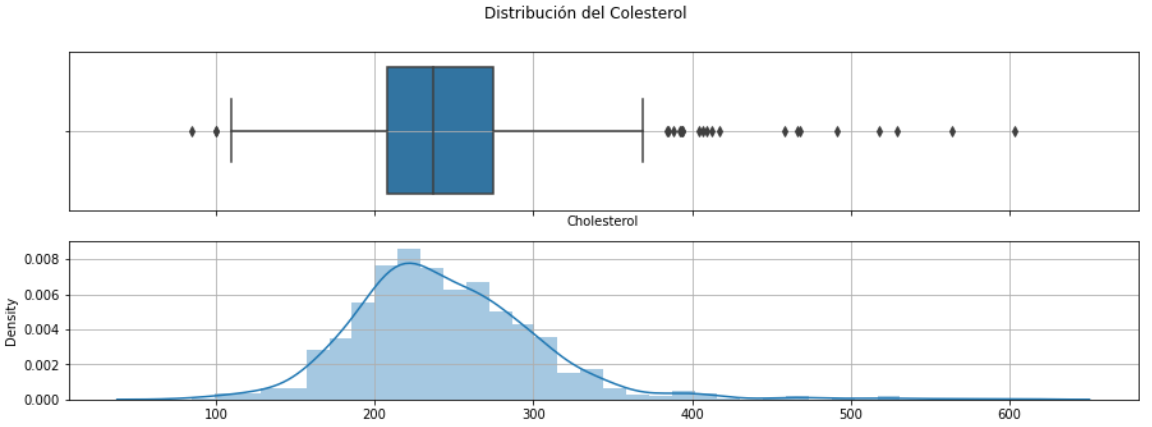


Figura 5: Distribución de la variable Colesterol.

Se observa que existen outliers debajo de la cota mínima y por encima de la cota máxima del boxplot. Redefiniremos su dominio al conjunto (100, 600), imputando únicamente 4 (cuatro) valores.

CODER HOUSE

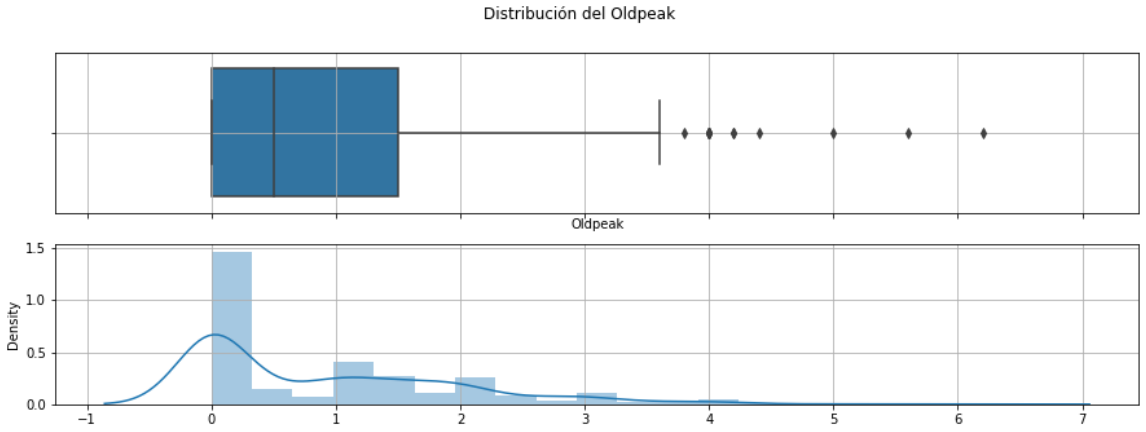


Figura 6: Distribución de la variable depresión de ST.

Se observan outliers sobre la cota superior del boxplot, pero todos se encuentran dentro del dominio , por lo cuál ninguno será imputado. El daset filtrado final a utilizar posee entonces ***741 filas y 12 columnas***, contando la variable target.

Correlación sin encodear (variables numéricas)

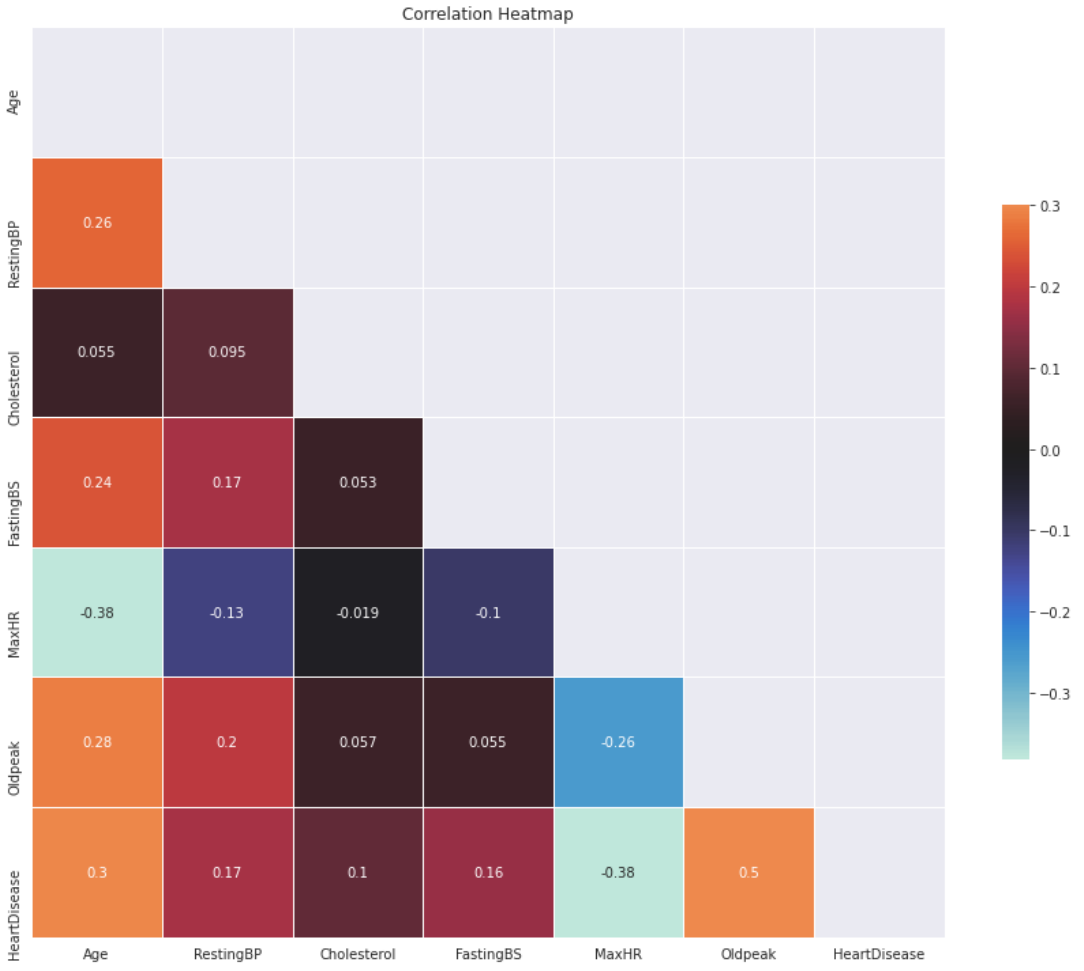


Figura 7: Correlación de Pearson utilizando solo variables numéricas.

CODER HOUSE

Puede observarse que -en módulo- las correlaciones de mayor valor se dan entre la variable target y la edad, la depresión de ST y el ritmo cardíaco máximo. Se brindará un análisis más exhaustivo en el notebook adjunto.

Feature Engineering

En una primera iteración, se dispuso a encodear las variables categóricas ordinales en enteros representativos y las nominales se binarizaron en 0 y 1, quedando el dataset de la siguiente manera:

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	0	3	140	289	0	1	172	0	0.0	1	0
1	49	1	2	160	180	0	1	156	0	1.0	2	1
2	37	0	3	130	283	0	2	98	0	0.0	1	0
3	48	1	1	138	214	0	1	108	1	1.5	2	1
4	54	0	2	150	195	0	1	122	0	0.0	1	0
...	...	...	...	...	...	...	...	...	...	...	...	...
913	45	0	4	110	264	0	1	132	0	1.2	2	1
914	68	0	1	144	193	1	1	141	0	3.4	2	1
915	57	0	1	130	131	0	1	115	1	1.2	2	1
916	57	1	3	130	236	0	3	174	0	0.0	2	1
917	38	0	2	138	175	0	1	173	0	0.0	1	0

741 rows × 12 columns

Figura 8: Dataset a utilizar en una primera iteración con variables categóricas encodeadas.

El dataset final a utilizar en la primera iteración cuenta con 354 filas correspondientes a personas con enfermedades coronarias y 387 filas correspondientes a personas sin éstas. Se dispuso a balancear las clases en el dataset, de modo tal de contar con las 354 filas de personas con enfermedades coronarias, y otras 354 filas elegidas al azar entre las 387 de personas sin las mismas. El dataset a utilizar para entrenar los modelos posee finalmente 708 filas y 12 columnas:

CODER HOUSE

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
268	54	0	1	130	242	0	1	91	1	1.0	2	1
202	42	0	2	160	147	0	1	146	0	0.0	1	0
71	44	0	3	130	215	0	1	135	0	0.0	1	0
602	61	0	2	120	337	0	1	98	1	0.0	2	1
95	58	0	1	130	263	0	1	140	1	2.0	2	1
...	...	...	...	...	...	...	...	...	...	...	...	...
97	39	0	2	160	147	1	1	160	0	0.0	1	0
189	53	0	1	180	285	0	2	120	1	1.5	2	1
51	47	1	1	120	205	0	1	98	1	2.0	2	1
669	45	1	3	130	234	0	3	175	0	0.6	2	0
761	52	0	1	112	230	0	1	160	0	0.0	1	1

708 rows × 12 columns

Figura 9: Dataset balanceado listo para utilizar en el modelado.

Correlación con variables encodeadas

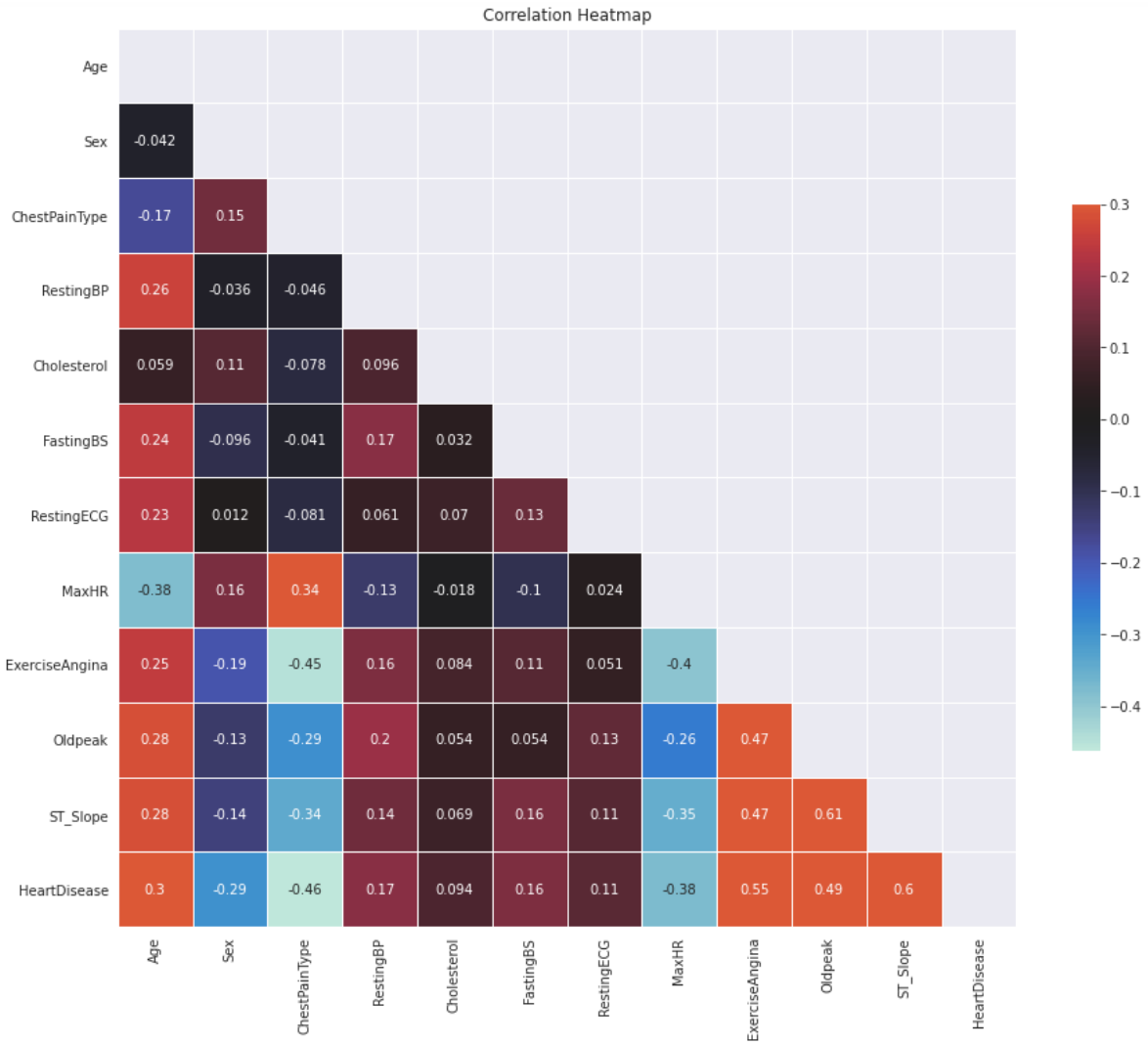


Figura 10: Correlación de Pearson utilizando todas las variables.

CODER HOUSE

Baseline

Como primera iteración se entrenará el modelo mediante un árbol de decisión clásico y un Random Forest sin optimizar hiperparámetros ni utilizar validación cruzada. Se generarán 3 (tres) sets de datos, uno para entrenar (train), otro para validar el modelo (val), y otro para testear el mejor de ambos. Se considera que el set de test no debe ser utilizado hasta haber elegido el mejor modelo con el set de validación.

Test - Train - Validation split

Se destinará un 80% del dataset para entrenamiento y un 20% para el testeo. El 10% del set de entrenamiento será destinado para el set de validación.

Métricas

El modelo que mejor performó fue el Random Forest con su configuración estándar, obteniéndose un **accuracy de 0.90[2] y un recall de 0.84** para la clase 1.

	precision	recall	f1-score	support
0	0.82	0.98	0.90	61
1	0.99	0.84	0.91	81
accuracy			0.90	142
macro avg	0.90	0.91	0.90	142
weighted avg	0.92	0.90	0.90	142

Figura 11: Classification report del modelo.

[2] **Fe de erratas**: Se corrigieron los valores de accuracy y recall obtenidos en la primera entrega, ya que no eran correctos. Los mismos pudieron comprobarse al correr nuevamente el notebook.

CODER HOUSE

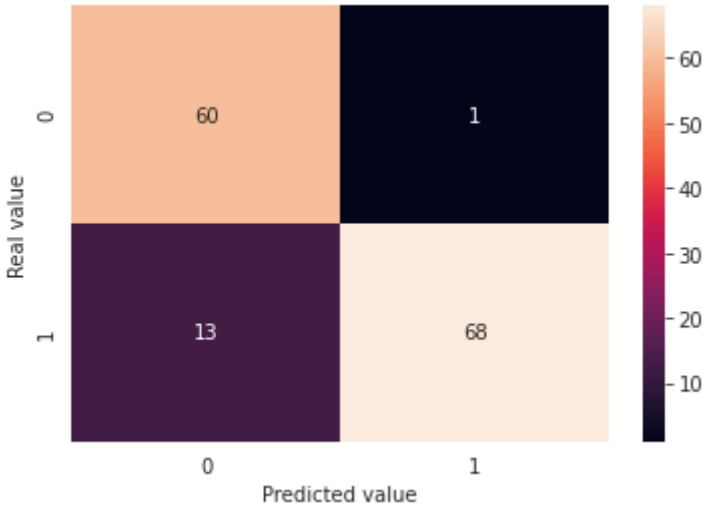


Figura 12: Matriz de confusión obtenida.

Puede observarse que los falsos positivos (FP), es decir, los valores que fueron predichos como positivos pero que en realidad no lo son, son menores a los falsos negativos (FN) que son los valores que fueron predichos como negativos pero en realidad son positivos. Si bien esto no condice exactamente con lo buscado para la clase 1, será mejorada en las próximas iteraciones.

Conclusiones primera iteración

El presente trabajo fue realizado bajo el concepto de estudio de factibilidad, realizando una única iteración sobre las variables elegidas, los motivos y la escueta selección de modelos. Aún así, se concluye que la clasificación es factible de ser realizada, habiendo alcanzado valores de métricas altos, susceptibles de ser utilizados en la medicina.

CODER HOUSE

Segunda iteración

Habiendo obtenido un baseline en la primera iteración que valida la hipótesis de factibilidad de la clasificación, se realizará una segunda iteración utilizando otros modelos y mejorando los hiperparámetros empíricamente de los ya utilizados buscando evitar el *overfitting* y obtener el mejor ***trade off*** sesgo-varianza posible para los más performantes. Se incluirán más métricas comparativas como la entropía cruzada binaria como función de pérdida, el área bajo la curva, y el gráfico de la curva ROC punto por punto, buscando optimizar la matriz de confusión en función de los requerimientos deseados, viendo si realmente el punto de corte por default para la clasificación binaria (>= 0.5 utilizando la función sigmoide) es realmente el punto de corte ideal que maximiza el TPR (recall), que - junto con el *accuracy*, son las dos principales métricas de negocio a optimizar.

Las nuevas métricas y modelos se evaluarán bajo la misma estratégia de encodeo y feature engineering utilizada en la primera iteración, trabajando sobre el dataset balanceado obtenido en la primera entrega. Esta estratégia puede cambiar en la próxima y última iteración, dependiendo de los resultados obtenidos en la presente entrega.

Modelos a implementar

Se probarán y optimizarán hiperparámetros sobre los siguientes modelos:

- **Árbol de decisión**: se decidió achicar la profundidad del árbol de 12 a 6, debido a que se encontraba copiando el set de entrenamiento, comportamiento que se comprobó mediante el accuracy y la función de pérdida utilizada, la cuál divergía considerablemente en el set de validación.

- **Random forest**: se optimizó la cantidad de estimadores a utilizar en función del estancamiento del accuracy en validación y la caída del mismo en train.

- **KNN**: se optimizó la cantidad de vecinos empíricamente hasta obtener un valor de accuracy que crezca en ambos sets (validación y entrenamiento).

- **Regresión logística**: se utilizó el modelo con los hiperparámetros brindados por default, para luego realizar una optimización de los mismos de forma automática mediante Gridsearch o equivalente en

una tercera iteración dada la variedad e importancia de cada uno de los mismos (warm_start, fit_intercept, entre otros).

Resultados obtenidos y comparativa de métricas

Función de pérdida sobre Train y Val

	Train	Val
Árbol de decisión	0.16	2.6
Random Forest	**0.11**	**0.27**
KNN	0.56	0.59
Regresión logística	0.39	0.28

Tabla 1: entropía cruzada obtenida sobre todos los sets de entrenamiento y validación.

Gracias a la entropía cruzada, es posible inferir -controlando previamente los hiperparámetros para evitar el *overfitting*- que el modelo que mejor capacidad de inferencia posee es el Random Forest con 40 estimadores. Es posible también observar como un Árbol de decisión talado respecto a la entrega anterior diverge considerablemente al inferir sobre un set de datos que no sea el de entrenamiento.

Accuracy sobre Train, Val y Test

	Train	Val	Test
Árbol de decisión	0.93	0.80	0.85
Random Forest	**0.99**	**0.93**	**0.92**
KNN	0.70	0.72	0.69
Regresión logística	0.83	**0.93**	0.87

Tabla 2: Exactitud obtenida sobre todos los sets.

CODER HOUSE

En esta tabla es posible observar los beneficios de haber optimizado los hiperparámetros sobre el set de validación. Se observa un incremento en la exactitud de todos los modelos utilizados respecto al baseline sobre el set de test, siendo el Random Forest nuevamente el que mejor performa.

Recall sobre Val y Test (clase 1)

	Val	Test
Árbol de decisión	0.88	0.80
Random Forest	**0.94**	**0.89**
KNN	0.70	0.57
Regresión logística	**0.94**	0.80

Tabla 3: TPR (Recall) obtenido sobre los sets de validación y testeo.

Se observa que nuevamente el Random Forest es el modelo que maximiza el Recall, siendo el modelo que mejor minimiza los falsos negativos.

ROC/AUC sobre Test incluyendo umbrales

En el notebook adjunto se incluye además las gráficas y los umbrales sobre el set de validación, así como también otras métricas como el F1-score que no se incluyeron en el presente informe por priorizar otras.

Árbol de decisión

CODER HOUSE

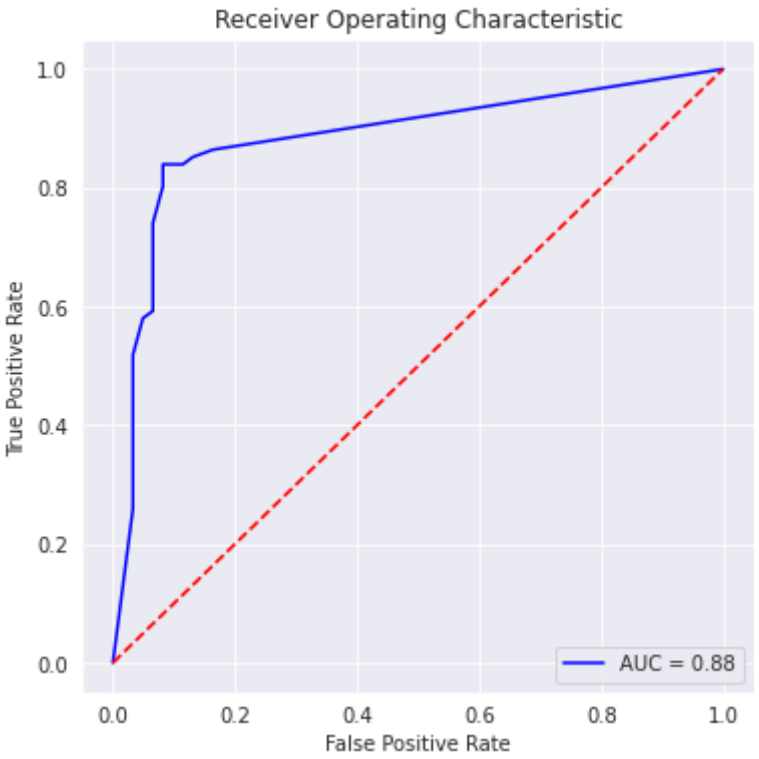


Figura 13: ROC/AUC sobre test utilizando el Árbol de decisión.

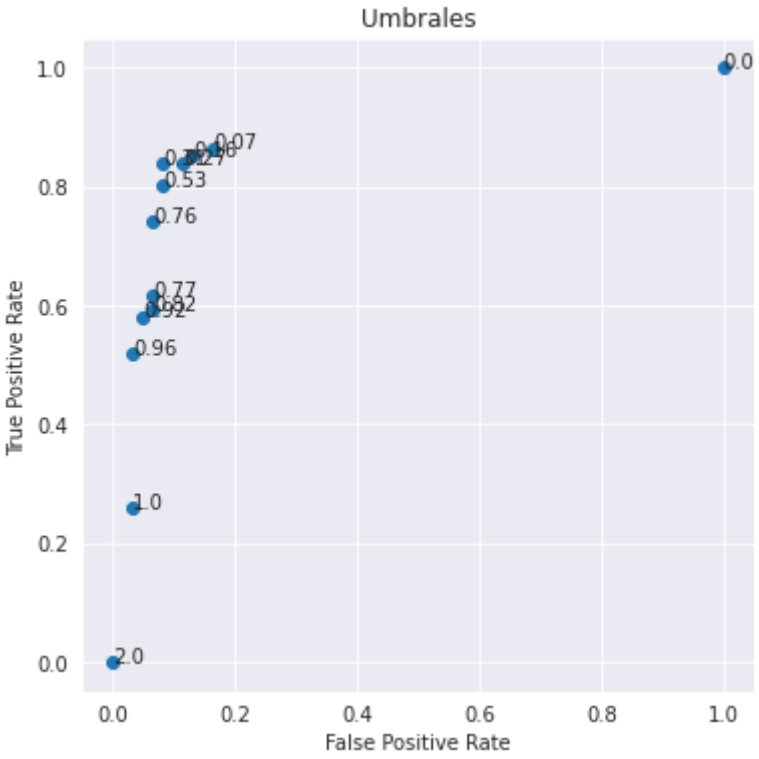


Figura 14: conjunto de umbrales correspondientes a todas las matrices de confusión de la ROC/AUC sobre test utilizando el Árbol de decisión.

CODER HOUSE

Random Forest

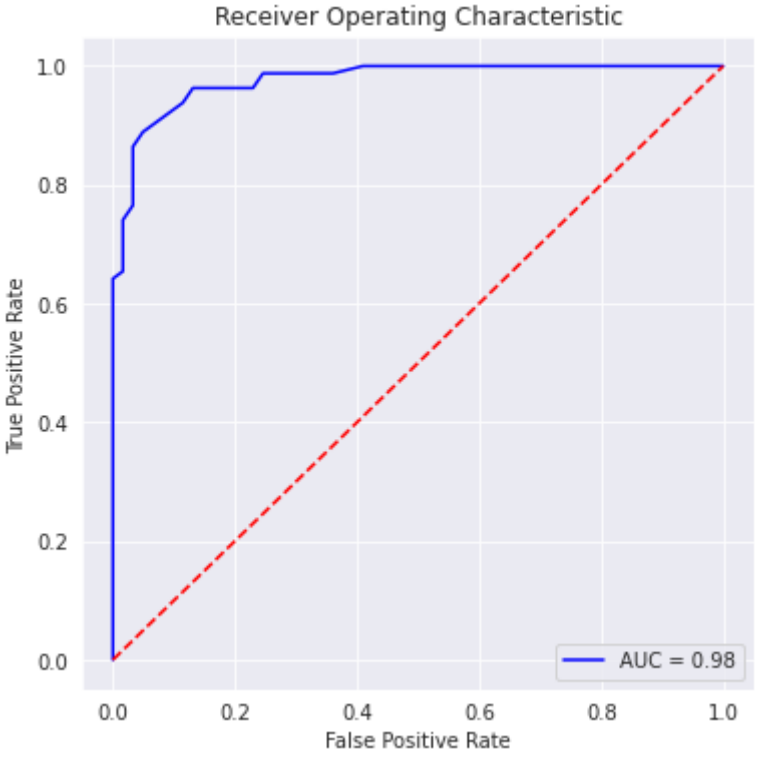


Figura 15: ROC/AUC sobre test utilizando Random Forest.

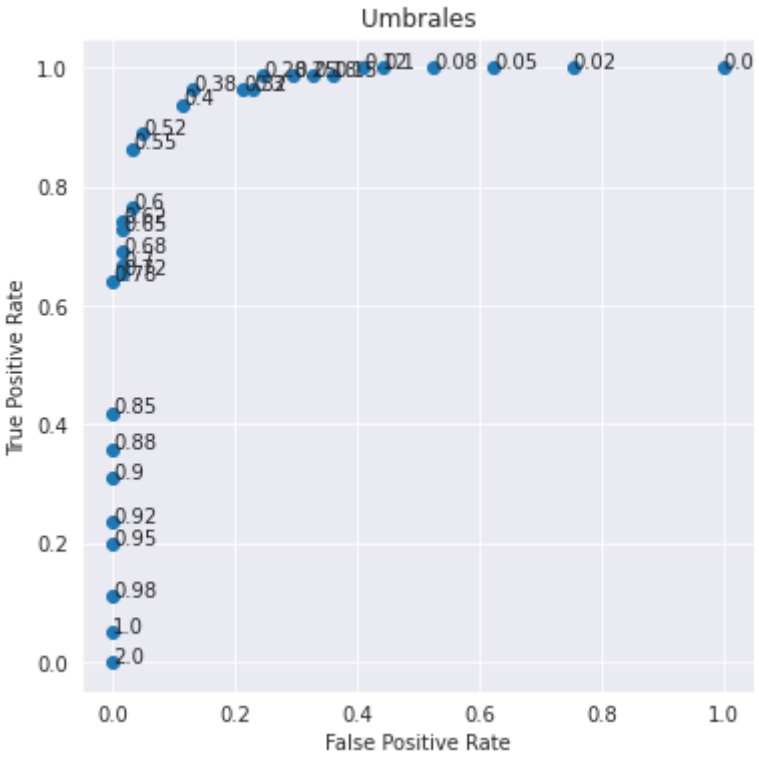


Figura 16: conjunto de umbrales correspondientes a todas las matrices de confusión de la ROC/AUC sobre test utilizando Random Forest.

CODER HOUSE

KNN

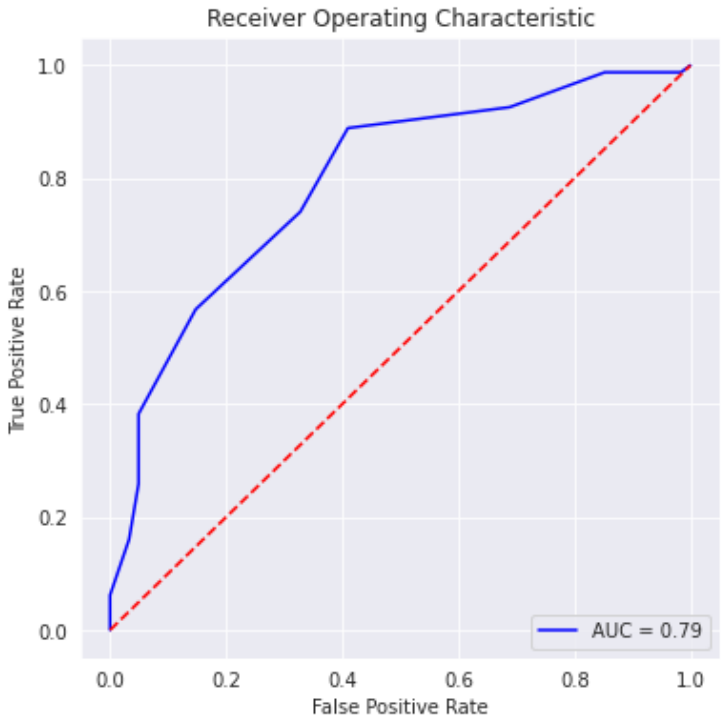


Figura 17: ROC/AUC sobre test utilizando KNN.

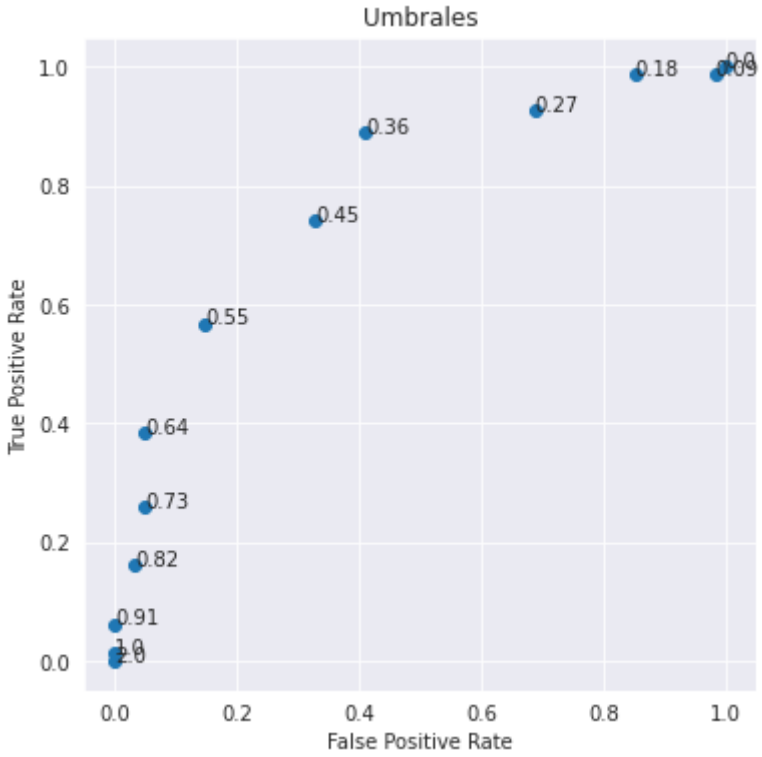


Figura 18: conjunto de umbrales correspondientes a todas las matrices de confusión de la ROC/AUC sobre test utilizando KNN.

CODER HOUSE

Regresión Logística

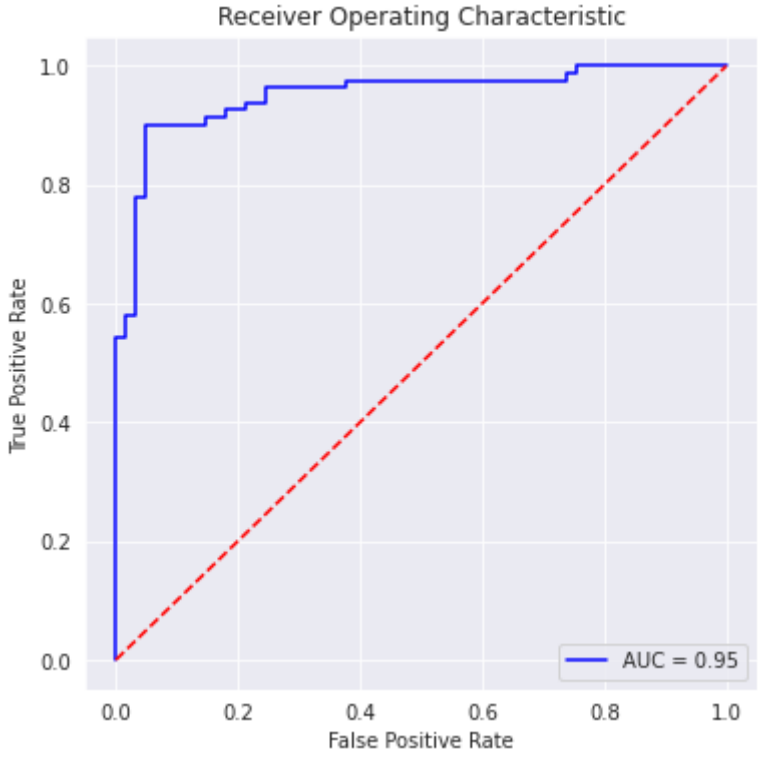


Figura 19: ROC/AUC sobre test utilizando Regresión logística.

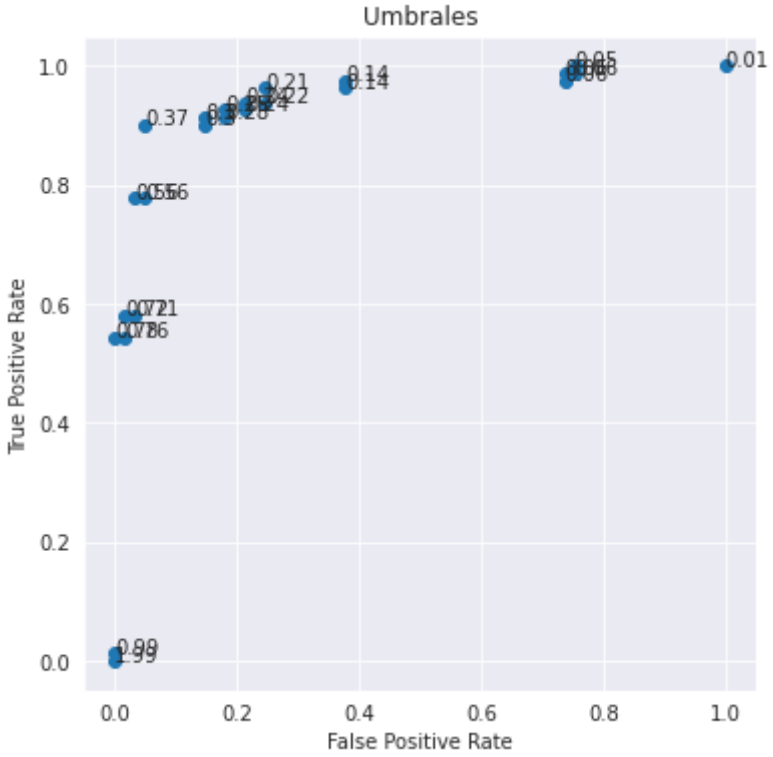


Figura 20: conjunto de umbrales correspondientes a todas las matrices de confusión de la ROC/AUC sobre test utilizando Regresión logística.

CODER HOUSE

Se observa -nuevamente- que la mejor performance a la hora de clasificar y distinguir la variable target viene dada por el Random Forest. Será propósito de otro apartado el obtener aquella matriz de confusión que maximice el TPR, siendo una de nuestras principales métricas de negocio, sin descuidar el accuracy.

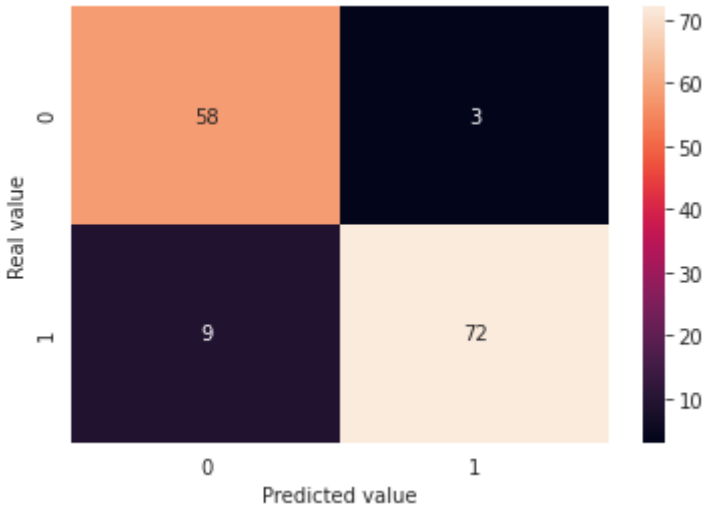


Figura 21: Matriz de confusión obtenida utilizando el modelo Random Forest mediante la optimización de estimadores.

Análisis PCA

Buscando optimizar los resultados obtenidos hasta el momento, se realizó una reducción de dimensionalidad, buscando como objetivo obtener el 95% de varianza en la menor cantidad de features o variables independientes posible, dando cómo resultado diez de las once variables presentes. Se probó este approach en los 2 modelos más performantes sobre el set de test: Random Forest y Regresión logística. Los resultados obtenidos fueron los siguientes:

	Accuracy	Recall	AUC
Random Forest	0.85	**0.81**	0.95
Regresión logística	**0.87**	0.80	**0.96**

Tabla 4: Métricas obtenidas mediante el análisis PCA.

CODER HOUSE

Puede visualizarse que la performance de ambos modelos bajo esta premisa es menor, por lo cuál se seguirá trabajando bajo la hipótesis principal, buscando mejorar las métricas existentes[3].

Corrección del umbral (optimal cutoff)

En la siguiente etapa se buscará -basándonos en el modelo que mejor performa- incrementar las métricas de negocio realizando una corrección al umbral definido por default intrínsecamente en el modelo al predecir. Se evaluarán aquellas matrices de confusión que maximicen el TPR (Recall) sin poner en desmedro el accuracy obtenido hasta el momento, pero teniendo en cuenta que un pequeño aumento en los falsos positivos no nos generaría una baja en la performance del modelo per se. Se analizará la curva ROC de Random Forest desde la derecha hacia la izquierda sobre el set de validación, buscando el umbral óptimo.

Trazando una analogía entre el TPR y el beneficio, como así también entre el FPR[4] y el costo, es posible generar una función susceptible de ser maximizada.

$$TPR - FPR \Rightarrow máx$$

	tpr	fpr	thr	tpr-fpr
16	0.939394	0.083333	0.525	0.856061
18	0.969697	0.166667	0.325	0.803030
15	0.878788	0.083333	0.600	0.795455
19	1.000000	0.208333	0.300	0.791667
14	0.818182	0.041667	0.625	0.776515

Figura 22: valor máximo obtenido de la función a maximizar.

Corrigiendo la clase predicha por este nuevo umbral de probabilidad, se obtienen métricas idénticas a las anteriores, tanto en el set de validación como en el set de test, indicando que es lo mismo para este set de valores, estimar la cota inferior en 0.5 que en 0.525.

[3] Se adjuntan más resultados, gráficos y conclusiones en el notebook que acompaña la entrega.
[4] FPR = FP / (FP + TN)

CODER HOUSE

Mejoras para iteraciones posteriores

A continuación se enuncian tres ideas para optimizar los resultados obtenidos:

1. Optimizar los hiperparámetros de los modelos que mejor performan de forma automática, mediante Gridsearch, abarcando más parámetros que los vistos en la presente entrega.

2. Implementar XGBoost con hiperparámetros optimizados.

3. Optimizar el flujo de trabajo del nosocomio ideando una matriz de confusión de 2 filas por 3 columnas, incorporando una semaforización ternaria mediante la parametrización de los umbrales, donde se puede estar enfermo, se puede ser un caso dudoso, o se puede estar sano, configurando un porcentaje destinado a la población dudosa.

Conclusiones segunda iteración

Se mejoraron todas las métricas obtenidas respecto al baseline presentado en la primera entrega aplicando la optimización manual de hiperparámetros logrando un **accuracy de 0.92 y un recall de 0.89** para la clase 1. Se comprobó a su vez que tanto una reducción en la dimensionalidad como un cambio en el umbral de cutoff no varían la performance del Random Forest, siendo éste el mejor modelo obtenido hasta el momento.

Modelo óptimo	Accuracy	Recall	AUC	F1
Random Forest	0.92	0.89	0.98	0.92

Tabla 5: Métricas óptimas obtenidas.

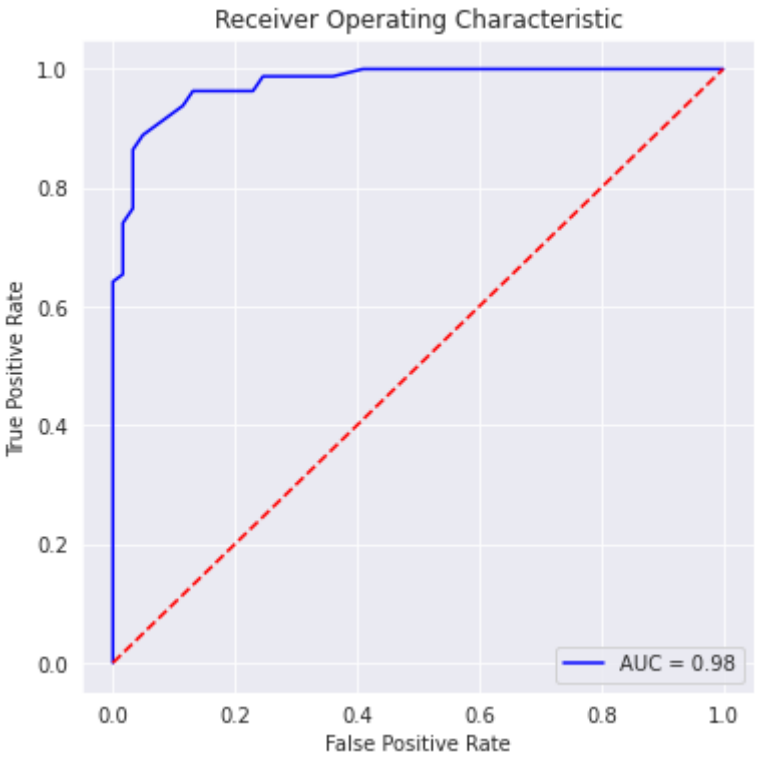


Figura 23: curva ROC obtenida para Random Forest .

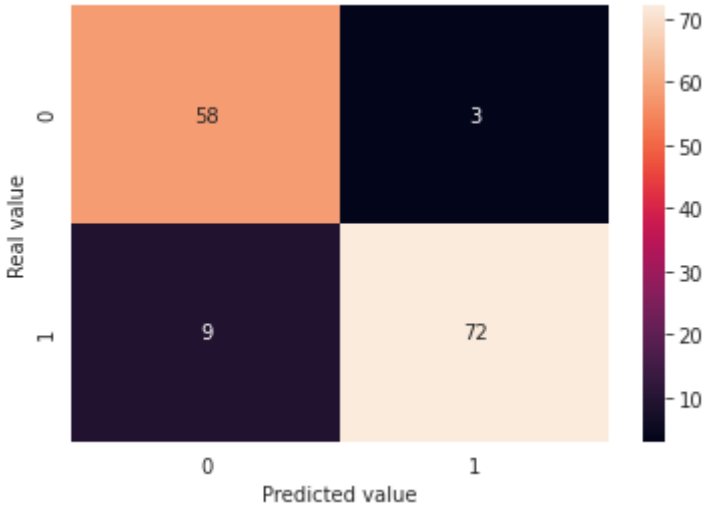


Figura 24: matriz de confusión obtenida.

CODER HOUSE

Tercera iteración

En esta tercera iteración se buscará mejorar la performance de los modelos utilizados en la anterior a través de la optimización de hiperparámetros a través de un del uso de validación cruzada mediante ***GridSearchCV*** y ***RandomizedSearchCV***. Se utilizará ***StratifiedKFold*** CV para evaluar los modelos optimizados, buscando que en cada iteración o fold haya la misma cantidad de elementos pertenecientes a ambas clases.

Se implementarán también modelos ensamble y boosting buscando mejorar las métricas logradas en iteraciones anteriores.

Modelos optimizados y métricas

GridSearchCV utilizando StratifiedKFold

Se fusionaron los sets de train y validación para la optimización de hiperparámetros ya que se va validando -valga la redundancia- con cada fold. Se obtuvieron los siguientes resultados sobre los modelos anteriormente analizados, utilizando 5 (cinco) folds y optimizando accuracy:

Árbol de decisión

Mejores hiparámetros obtenidos: *{'ccp_alpha': 0.0, 'criterion': 'entropy', 'max_depth': 4, 'min_samples_leaf': 9, 'min_samples_split': 2}*

Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
0.826	0.043	**0.869**	**0.845**	**0.765**

Tabla 6: Métricas obtenidas mediante la optimización de parámetros.

CODER HOUSE

Puede observarse en el notebook adjunto que las métricas a lo largo de los cinco splits se mantuvieron consistentes, generando un modelo robusto con un pequeño desvío estándar. Las métricas alcanzadas no son suficientes para reemplazar el mejor modelo obtenido, ya que su performance es menor al mismo sobre datos desconocidos.

Random Forest

Mejores hiparámetros obtenidos: {'bootstrap': False, 'max_depth': 5, 'max_features': 'auto', 'min_samples_leaf': 1, 'min_samples_split': 10, 'n_estimators': 21}

Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
0.873	0.018	**0.910**	**0.873**	**0.839**

Tabla 7: Métricas obtenidas mediante la optimización de parámetros.

El modelo es consistente con las métricas y -bajo esta configuración de hiperparámetros- *es el que mejor performó sobre datos desconocidos teniendo en cuenta dos métricas importantes de negocio (Accuracy y Recall). No obstante, éstos valores se encuentran alejados de los obtenidos en la iteración anterior*.

KNN

Mejores hiparámetros obtenidos: {'metric': 'manhattan', 'n_neighbors': 11, 'weights': 'distance'}

Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
0.722	0.013	**1.0**	**0.760**	**0.716**

Tabla 8: Métricas obtenidas mediante la optimización de parámetros.

CODER HOUSE

Si bien el modelo es constante con las métricas en cada fold, obteniendo un bajo desvío estándar, obtiene bajos resultados a la hora de generalizar sobre datos desconocidos.

Regresión Logística

Mejores hiparámetros obtenidos: {'C': 3.727593720314938, 'penalty': 'l2', 'warm_start': False}

Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
0.845	0.020	0.858	**0.866**	**0.802**

Tabla 9: Métricas obtenidas mediante la optimización de parámetros.

El modelo performa correctamente sobre el set de test, y continúa siendo el segundo mejor modelo a elegir para resolver esta clasificación. Las métricas obtenidas mediante esta técnica de optimización de hiperparámetros *arrojó resultados levemente superiores a la iteración anterior, pero sin superar al Random Forest.*

RandomizedSearchCV utilizando StratifiedKFold

Modelo	Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
Árbol de decisión	0.810	0.009	0.810	**0.894**	**0.901**
Random Forest	**0.857**	**0.002**	**0.878**	0.873	0.839
KNN	0.715	0.019	1.0	0.760	0.753
Regresión Logística	0.845	0.02	0.858	0.866	0.802

Tabla 10: Métricas obtenidas mediante la optimización de parámetros utilizando RandomizedSearchCV.

CODER HOUSE

Utilizando RandomizedSearchCV puede observarse que las mejores métricas sobre el set de test fueron obtenidas con un modelo que no era performante hasta el momento, como ser el Árbol de decisión, obteniendo inclusive una centésima más de Recall que el mejor modelo evaluado en esta presentación. El problema que presentan las métricas de este modelo es que el Accuracy obtenido en el set de test es sustancialmente superior al obtenido en el set de train. Esto puede deberse al pequeño desbalanceo presente en las distribuciones de clases en el set de test, donde predomina la clase 1, y -por lo tanto- no es un resultado representativo y no será tenido en cuenta. Se adjuntan en el notebook los hiperparámetros obtenidos para cada uno de los modelos.

Modelos boosting

Buscando mejorar las métricas obtenidas hasta el momento, se implementarán 4 (cuatro) modelos de boosting utilizando sus parámetros por default. Los valores obtenidos fueron los siguientes:[5]

Modelo	Acc train	Acc val	Acc test	Recall val (clase 1)	Recall test (clase 1)	Entropía cruzada train	Entropía cruzada val	AUC test
AdaBoost	0.87	0.86	0.87	0.82	0.80	0.66	0.64	0.95
Gradient Boosting	**0.96**	**0.91**	0.88	**0.91**	**0.85**	0.17	0.25	**0.98**
XGBoost	1.0	0.86	0.86	0.91	0.85	0.02	0.31	0.95
LightGBM	1.0	0.86	**0.89**	0.88	0.84	0.02	0.31	0.95

Tabla 11:Comparativa de métricas entre modelos boosting implementados.

De los modelos analizados, resalta por su performance sobre todos los indicadores el ***Gradient Boosting***. Si bien el modelo minimiza la entropía cruzada entre train y validación y obtiene métricas más que aceptables, no es capaz de superar al Random Forest obtenido en la iteración anterior.

[5] Las curvas ROC, los umbrales y las matrices de confusión se adjuntan en el notebook para no sobrecargar la presentación.

CODER HOUSE

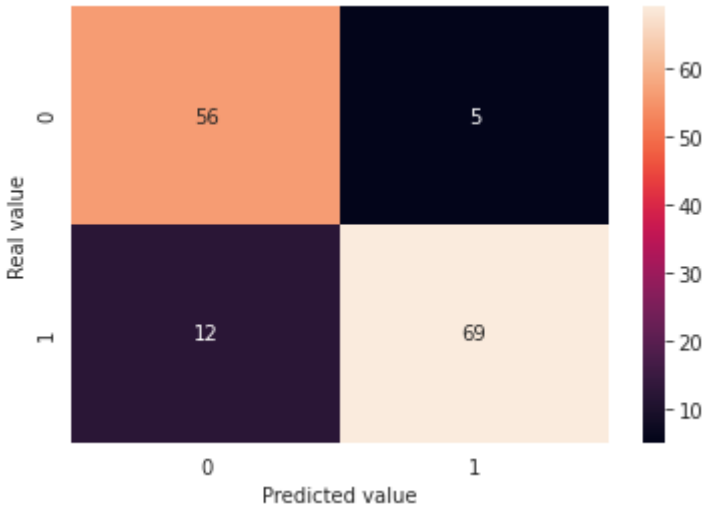


Figura 25: matriz de confusión obtenida mediante Gradient Boosting sobre el set de test.

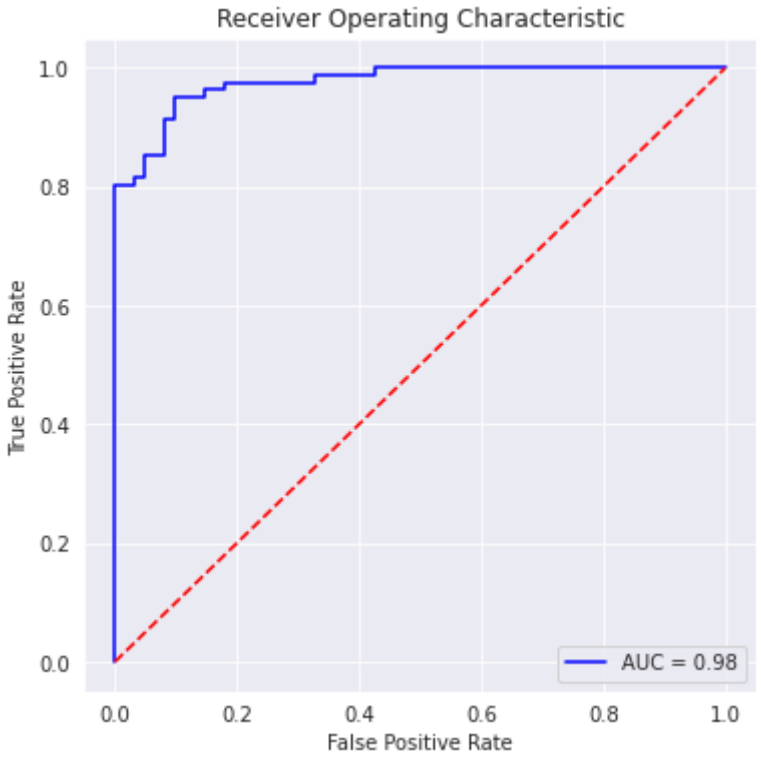


Figura 26: curva ROC obtenida para Gradient Boosting sobre el set de test.

Puede observarse en la figura 25 que tanto el Accuracy, el Recall y Precision son menores que el modelo obtenido anteriormente.

CODER HOUSE

Modelos boosting optimizados

Se optimizaron los 4 (cuatro) modelos utilizados anteriormente mediante RandomizedSearchCV utilizando StratifiedKFold. Se buscará generar una grilla de parámetros lo más variada posible. Los resultados obtenidos fueron los siguientes:

Modelo	Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
AdaBoost	0.831	0.042	0.825	0.823	0.728
Gradient Boosting	0.838	0.037	0.876	0.859	0.790
XGBoost	**0.844**	**0.031**	**0.919**	**0.894**	**0.851**
LightGBM	0.830	0.041	0.858	0.880	0.827

Tabla 12: Métricas obtenidas mediante la optimización de parámetros utilizando RandomizedSearchCV sobre los modelos boosting.

De los modelos optimizados, resalta por su performance sobre todos los indicadores el ***XGBoost***. Si bien el modelo obtenido performa muy bien, no es capaz de superar al Random Forest obtenido en la iteración anterior.

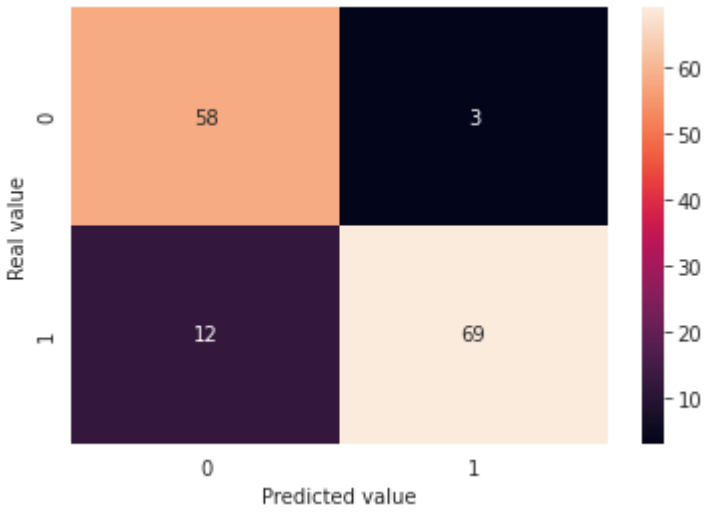


Figura 27: matriz de confusión obtenida mediante XGBoost optimizado sobre el set de test.

CODER HOUSE

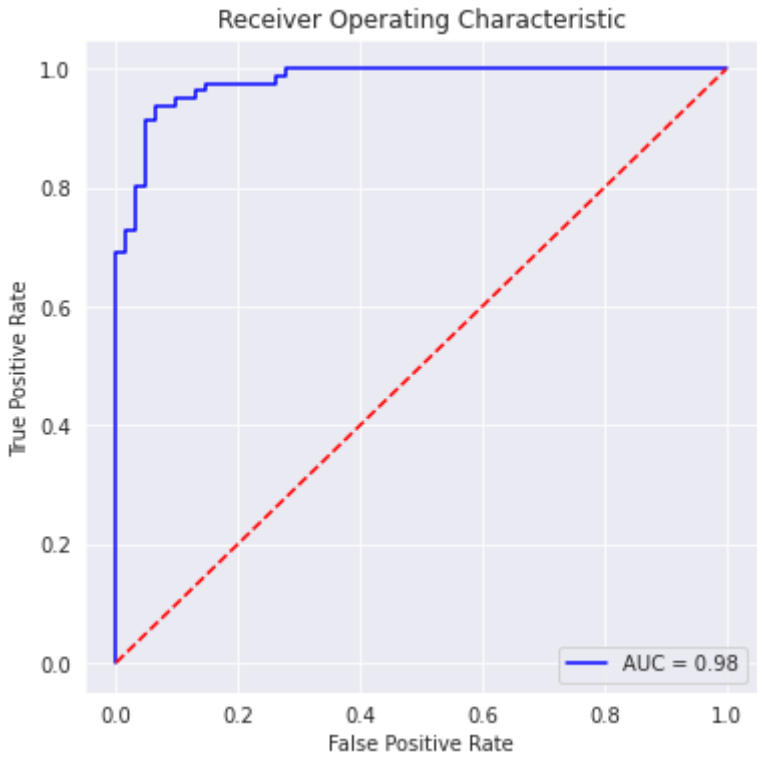


Figura 28: curva ROC obtenida para XGBoost optimizado sobre el set de test.

Puede observarse en la figura 27 que tanto el Accuracy como el Recall son menores que el modelo obtenido en la iteración anterior.

Bagging (Bootstrap aggregating)

Se implementará un **BaggingClassifier** con árboles de decisión debido a que son los modelos que más varianza o inestabilidad presentan. Se buscará el número de estimadores ideal empíricamente, evitando que el modelo overfiteé. Los resultados obtenidos fueron los siguientes:

Modelo	Acc train	Acc val	Acc test	Recall val (clase 1)	Recall test (clase 1)	Entropía cruzada train	Entropía cruzada val	AUC test
Bagging Classifier	0.99	0.88	**0.87**	0.88	**0.84**	0.11	0.28	**0.97**

Tabla 13: Métricas obtenidas mediante la optimización de parámetros utilizando RandomizedSearchCV sobre los modelos boosting.

CODER HOUSE

Nuevamente, el modelo obtenido no logra superar las mejores métricas de negocio obtenidas hasta el momento.

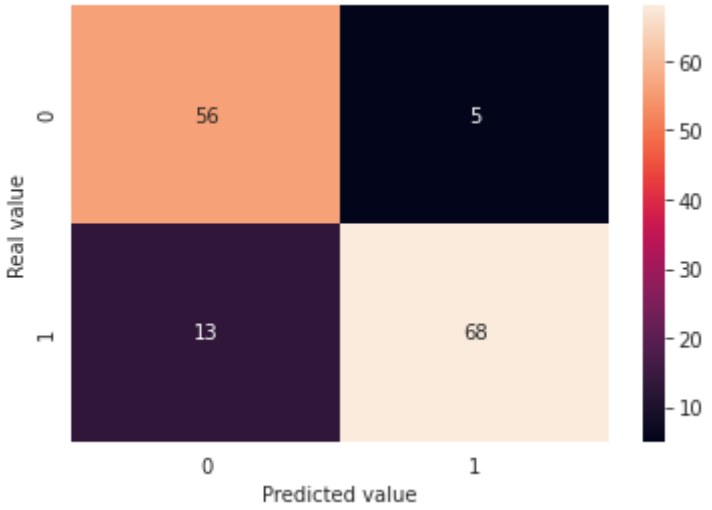


Figura 29: matriz de confusión obtenida mediante Bagging Classifier sobre el set de test.

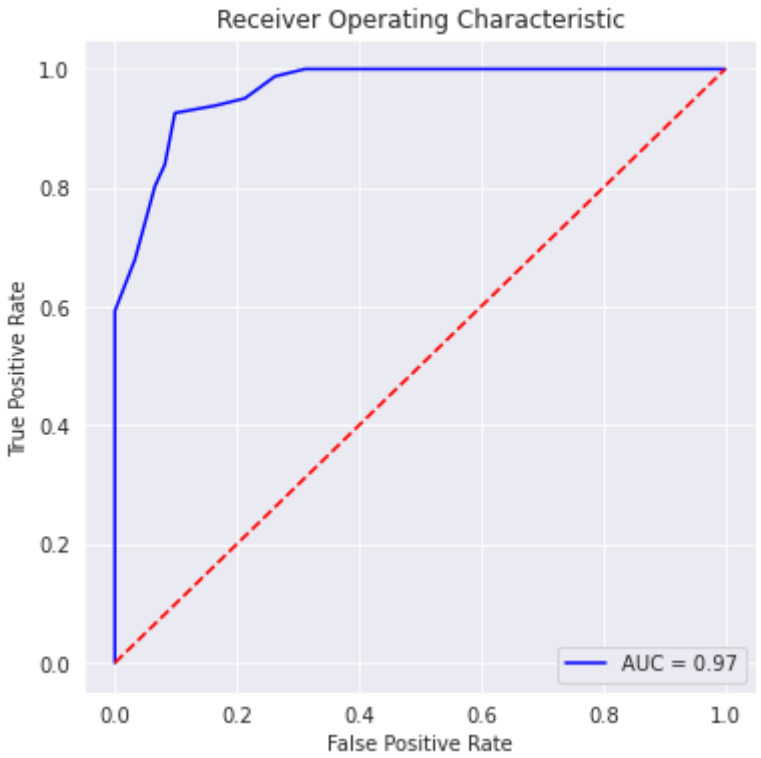


Figura 30: curva ROC obtenida para Bagging Classifier sobre el set de test.

CODER HOUSE

Bagging optimizado

Se optimizarán los hiperparámetros tanto del estimador base utilizado (Árbol de decisión) como del Bagging Classifier propiamente dicho mediante el uso de ***RandomizedSearchCV*** en conjunto con ***StratifiedKFold***.

Modelo	Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
Bagging Classifier	0.842	0.033	0.924	0.866	0.802

Tabla 14: Métricas obtenidas mediante la optimización de parámetros utilizando RandomizedSearchCV sobre Bagging Classifier y su estimador base.

El modelo optimizado performa de manera muy similar al optimizado empíricamente, con la salvedad de que se ve penalizada levemente una métrica importante de negocio como lo es el Recall o TPR, lo cuál puede visualizarse en la matriz de confusión adjuntada:

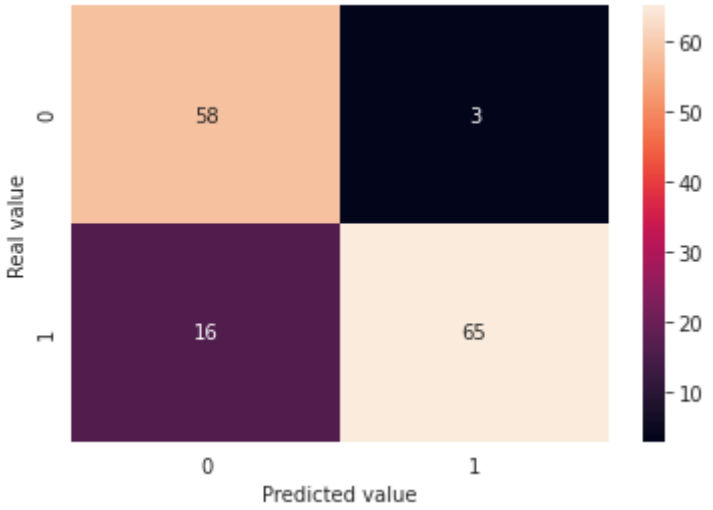


Figura 31: matriz de confusión obtenida mediante Bagging Classifier optimizado sobre el set de test.

CODER HOUSE

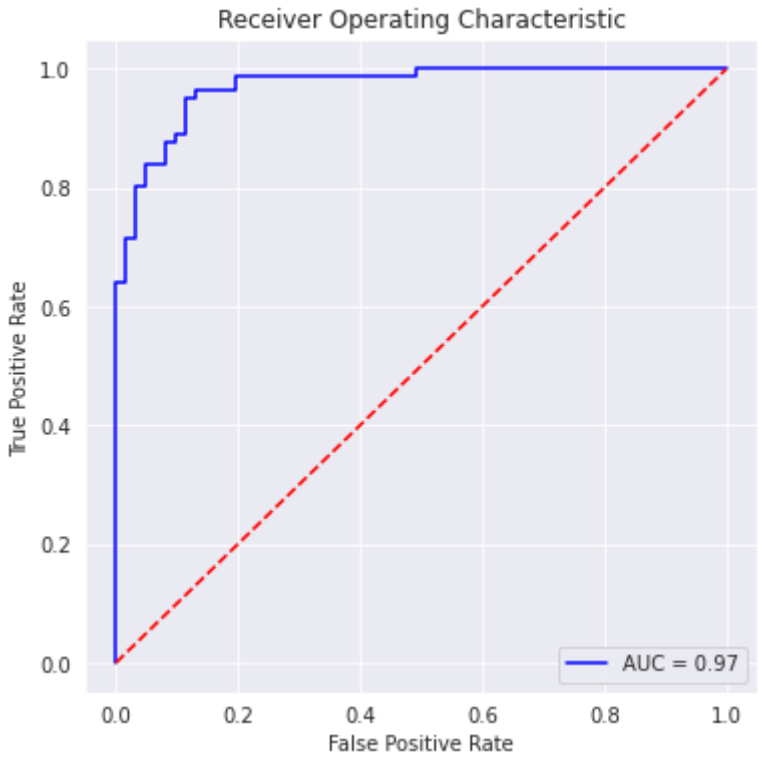


Figura 32: curva ROC obtenida para Bagging Classifier optimizado sobre el set de test.

Conclusiones tercera iteración

En base a los modelos analizados y optimizados presentes en este informe, si bien se obtuvieron resultados aceptables mediante los approaches utilizados en la tercera entrega, es posible concluir que el modelo que mejor performa sobre datos desconocidos es el ***Random Forest*** optimizado empíricamente en la segunda iteración, cuyas métricas se encuentran en las conclusiones de la misma.

Se logró obtener un modelo que maximiza con valores altos las métricas de negocio pedidas. No obstante, si las mismas no fueran suficientes para el cliente, es posible en una próxima iteración umbralizar la matriz de confusión de forma tal de generar un porcentaje de “dudosos”, buscando minimizar aún más los falsos negativos.

Modelo óptimo	Accuracy	Recall	AUC	F1
Random Forest	0.92	0.89	0.98	0.92

Tabla 15: Métricas principales óptimas obtenidas sobre el set de test.

CODER HOUSE

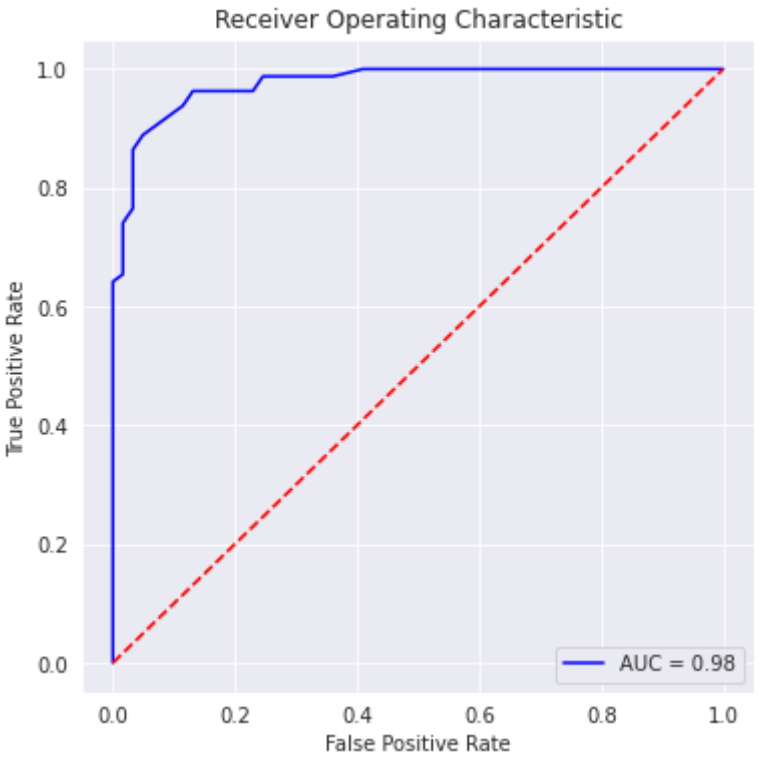


Figura 33: curva ROC óptima obtenida sobre el set de test .

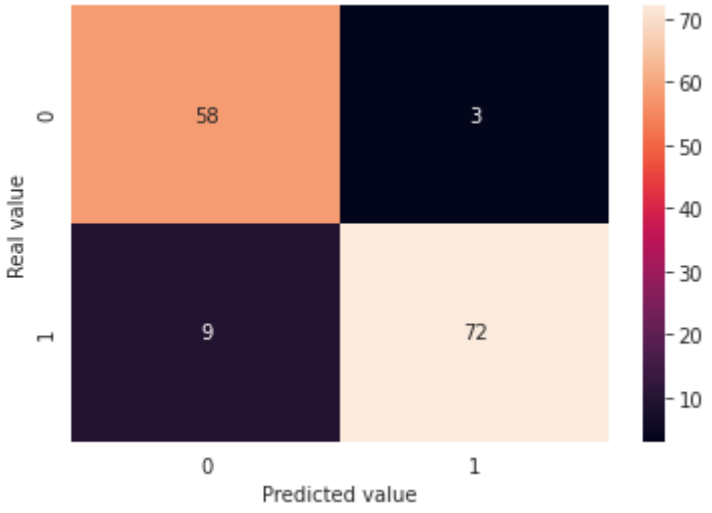


Figura 34: matriz de confusión óptima obtenida sobre el set de test.

CODER HOUSE

Conclusiones finales

Se logró desarrollar y obtener un modelo robusto, validado a través de tres iteraciones y el desarrollo adicional de diferentes modelos y técnicas. Se recomienda a su vez continuar con la recolección de datos no solo para mejorar la robustez del mejor modelo generado en este trabajo sino también para validarlo y/o cambiar sus umbrales a futuro en caso de ser necesario. Bajo esta premisa, se recomienda deployar el modelo en modo “shadow” persistiendo en la base de datos los resultados obtenidos hasta obtener observaciones suficientes como para evaluar su performance. Se recuerda también que -de desearlo- el nosocomio cuenta con la posibilidad de trabajar bajo una semaforización ternaria, buscando optimizar los recursos del mismo y entendiendo que su éxito radicará en la mayor detección posible de enfermedades coronarias con la menor cantidad de estudios complejos y/o costosos.

Ejemplo de semaforización ternaria:

Predicciones / Observaciones	Clase -1	Clase 0	clase 1
Clase -1	TP	PFN	FN
Clase 1	FP	PFP	TN

Tabla 16: Matriz de confusión adaptada para una semaforización ternaria.

- Clase -1: clase observada o predicha como un ***individuo con una enfermedad coronaria***.

- Clase 1: clase observada o predicha como un individuo que ***no posee*** una enfermedad coronaria.

- Clase 0: ***clase predicha*** cuyos umbrales de clasificación adaptados ***generan una población de dudosos***, los cuales pueden ser interpretados como pseudo falsos positivos (PFP) y pseudo falsos negativos (PFN). Idealmente, esta clase será la única a ser sujeta a estudios adicionales para verificar su condición, pudiendo incluso seleccionar estudios a aplicar según su probabilidad de clase.